

Module 2: Data Types

Introduction to C++ Programming — A Beginner's Course

Module Overview

This module's real payload is one specific surprise: integer division. Everything else (int, double, char, bool, std::string, sizeof) is vocabulary-building, but the moment students see `7 / 2` print 3 instead of 3.5 is the moment 'types aren't just labels, they change behavior' actually lands. Structure the session to build up to that moment rather than mentioning it in passing.

Suggested Duration: 60 minutes

Learning Objectives

By the end of this module, students will be able to:

- Choose an appropriate built-in type for a given piece of data.
- Explain why char uses single quotes and std::string uses double quotes.
- Predict the result of int/int division vs. division involving at least one double.
- Use `static_cast<double>(...)` to force a floating-point result.
- Use `sizeof` to inspect how much memory a type reserves.

Prerequisites

- Module 1: comfortable compiling/running programs and reading basic errors.

Materials & Instructor Prep

- `02-data-types/examples/03_integer_division_surprise.cpp` preloaded for a live demo.

Key Vocabulary

Term	Definition
Data type	An agreement about how many bits to reserve and how to interpret them (e.g. as a whole number, a letter, true/false).
Literal	A value written directly in code: <code>42</code> , <code>3.14</code> , <code>'A'</code> , <code>"hello"</code> , <code>true</code> .

Term	Definition
Type conversion / casting	Converting a value from one type to another, implicitly (automatic) or explicitly (static_cast).
Truncation	Cutting off a decimal portion rather than rounding, e.g. storing 9.8 into an int gives 9.

Lesson Outline & Instructor Talking Points

1. Why types exist at all (10 min)

- Frame it concretely: 'A computer only ever stores 0s and 1s. A type is the agreement about what those bits mean — a number, a letter, a price.'
- Introduce the table of core types (int, double, float, char, bool, std::string) as a reference, not something to memorize by rote yet — they'll internalize it through use.

2. Single vs. double quotes — a rule with a real reason (10 min)

- Show 'A' vs "A" side by side. State plainly: single quotes hold exactly one character; double quotes hold text of any length, including one character or zero.
- Predict-then-run: what happens if you write char c = "A"; (compiler error) vs std::string s = 'A'; (also an error, different reason).

Instructor note: *This single-vs-double-quote rule is one of the most persistent beginner slip-ups for the rest of the course — over-invest time here now.*

3. THE integer division surprise (20 min) — build to this deliberately

- Put 7 / 2 on the board. Ask students to predict the answer before running anything. Most will say 3.5.
- Run examples/03_integer_division_surprise.cpp live. Let the '3' land as a genuine surprise.
- Explain: when both operands are int, C++ performs integer division — the decimal part is discarded, not rounded.
- Introduce static_cast<double>(a) / b as the fix, and connect it back to Module 1's 'compiler errors are useful' framing: this isn't an error at all, which is precisely why it's dangerous — it compiles fine and gives a silently wrong answer.

Instructor note: *This is the single highest-value 20 minutes in the module. If time runs short elsewhere, protect this section rather than cutting it.*

4. sizeof as a concrete window into memory (10 min)

- Run examples/02_sizeof.cpp live. Ask: 'Why does bool only need 1 byte when it only stores two possible values?' (answer: memory is allocated in byte-sized chunks at minimum — there's no sub-byte allocation in C++)

Instructor note: Keep this light — sizeof itself is rarely used directly in student code going forward; its value here is purely conceptual (types have real memory cost).

5. std::string as a preview (10 min)

- Note explicitly that std::string isn't a built-in language type like the others — it needs #include <string>. Flag this now so the 'forgot the include' error is recognizable later.

Common Student Misconceptions

Misconception	How to Address It
"int / int rounds to the nearest whole number."	It truncates (discards the decimal), which is not the same as rounding — 7/2 gives 3, but so would (a hypothetical) 7.9/2 → the decision happens before any rounding logic exists.
"'A' and 'A'" are interchangeable."	Single quotes = exactly one char; double quotes = a string, even if it's one character long. They are different types with different capabilities.
"static_cast<double>(a) changes the variable a permanently."	Clarify it only affects the value in that expression — a itself is untouched unless you explicitly reassign it.

Exercises & Module Project

Exercise: 02-data-types/exercises/exercise1.cpp — declare each core type and compute a total price. Project: a temperature converter (02-data-types/project/README.md) that deliberately breaks itself by swapping double for int, to force students to witness the truncation bug in a project they built.

Facilitation guidance: Do not skip the project's 'deliberately break it' step even if pressed for time — it is the single most memorable moment in the module because students cause and then explain the bug themselves, rather than just watching a demo.

Discussion & Comprehension Check Questions

1. Why does 7 / 2 give 3 in C++ instead of 3.5?

Guidance: Both operands are int, so C++ performs integer division and truncates the decimal portion.

2. What's one situation where you'd deliberately want integer division's truncating behavior?

Guidance: e.g. computing how many full groups of N fit into a total (minutes into whole hours, items into full boxes).

Assessment Ideas

- Cold-call prediction quiz: put 3–4 arithmetic expressions with mixed types on the board (e.g. $9 / 4$, $9.0 / 4$, $9 \% 4$) and have students write down predicted outputs before revealing.

Answer Key Notes

See `02-data-types/exercises/solutions/exercise1_solution.cpp` — check specifically that students used `static_cast` correctly if their exercise required a decimal result.

Wrap-Up & Bridge to Next Module

Bridge forward: "Today every value lived for exactly one line. Next module we give values names that persist — variables — and start reading real input from the user instead of only printing fixed text."

Instructor Tips

- If you only have time to nail one concept this module, make it integer division — it resurfaces silently in student bugs for the rest of the course if not internalized now.